Programação Concorrente e Distribuída

Sockets

Glêdson Elias
gledson@ci.ufpb.br

1

Introdução

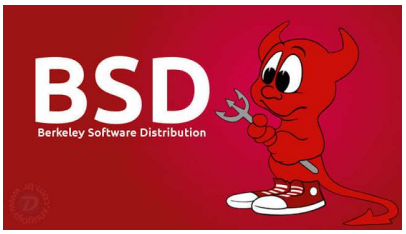
- Objetivo
  - Aprender a desenvolver aplicações distribuídas para Internet com arquitetura cliente-servidor que se comunicam via sockets



2

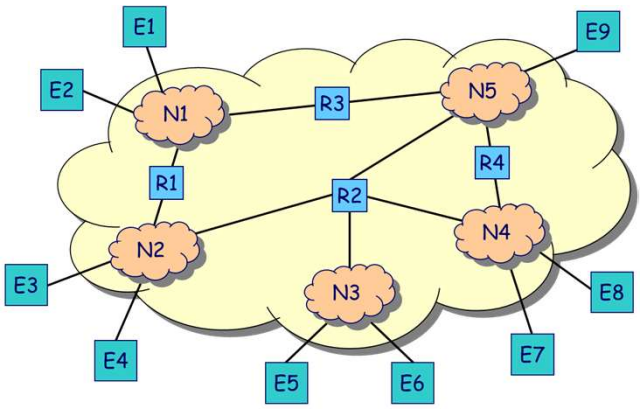
Introdução

- Socket
  - Interface de programação que possibilita a comunicação entre processos
    - Originalmente proposta no sistema operacional Unix BSD para linguagem C
    - Pode ser encontrado na literatura como Berkeley Sockets
  - Popularizou-se nos diversos sistemas operacionais e linguagens de programação



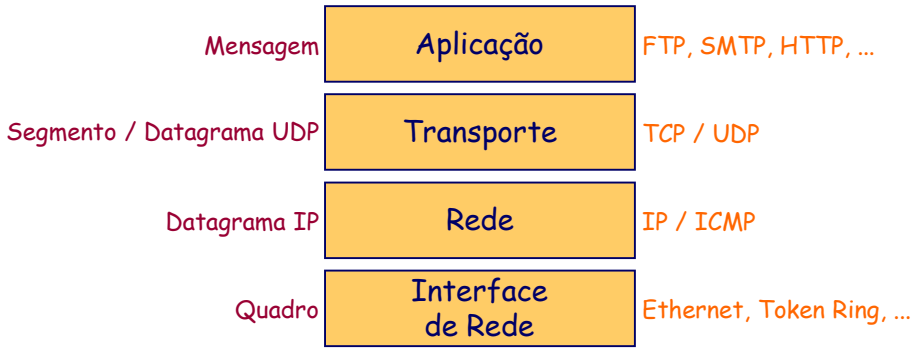
Arquitetura TCP/IP

- Modelo de Interconexão



Arquitetura TCP/IP

- Arquitetura em Camadas



Arquitetura TCP/IP

- Endereçamento
  - Endereço IP
    - Identifica cada rede e dispositivo da internet
    - Adota esquema de endereçamento hierárquico
    - Representado com 32/128 bits
      - IPv4: 192.168.10.1
      - IPv6: 2001:0db8:ad1f:25e2:cade:cafe:f0ca:84c1
  - Porta
    - Identifica localmente um ponto de comunicação no sistema operacional
    - Representado com número inteiro de 16 bits
    - Processos são associados a portas

Arquitetura TCP/IP

- Serviços de Transporte
  - Protocolo UDP
    - Provê serviço de datagrama não confiável e sem conexão
    - Adotado para aplicações que demandam entrega mais rápida e/ou suportam a perda de mensagens
      - DNS, VoIP, IPTV, ...
  - Protocolo TCP
    - Provê serviço de circuito virtual confiável e orientado a conexão
    - Provê um canal ponto-a-ponto para aplicações que demandam comunicação confiável
      - HTTP, FTP, Telnet, ...

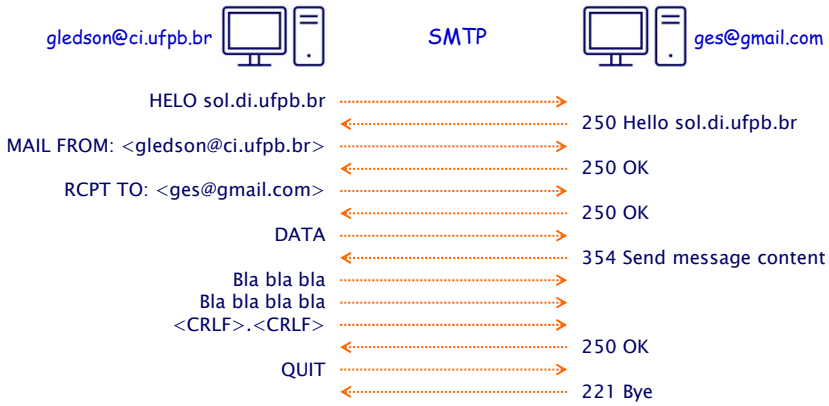
Arquitetura TCP/IP

- Protocolo de Aplicação
  - Define a sintaxe e a semântica das mensagens trocadas entre aplicações
  - Única camada cuja implementação é realizada usando processos do sistema operacional

Telnet	Terminal virtual
FTP	Transferência de arquivos
SMTP	Correio eletrônico
DNS	Serviço de nomes
HTTP	WWW (World Wide Web)

Arquitetura TCP/IP

- Protocolo de Aplicação



Arquitetura TCP/IP

- Desenvolvimento de Aplicações
  - Processos usam os serviços providos pela camada de transporte
    - Desenvolvedor decide qual serviço da camada de transporte é mais adequada às necessidades da aplicação
  - Processos interagem com implementações dos protocolos TCP e UDP através de uma API (Application Programming Interface)
  - Sockets representam um dos principais exemplos de interface de interação

Cliente-Servidor

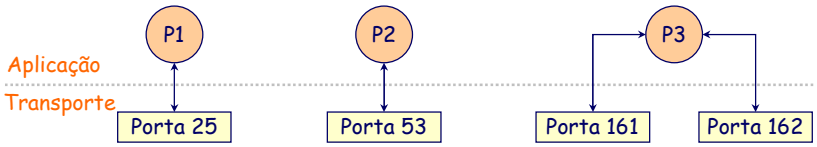
- Modelo Cliente/Servidor
  - Cliente
    - Requisita um serviço oferecido por um servidor
    - Inicia a interação com o servidor
    - Disponibiliza uma interface para o usuário
    - Finaliza execução após ser utilizado pelo usuário
  - Servidor
    - Oferece um serviço que pode ser requisitado pelos clientes através da rede
    - Comunica-se com o cliente somente após receber uma requisição
    - Executa continuamente

Cliente-Servidor

- Paradigma Requisição/Resposta
  - Cliente
    - Envia requisições para um ou vários servidores
    - Aguarda o recebimento das respectivas respostas
  - Servidor
    - Aceita requisições de um ou vários clientes
    - Provê serviço realizando o processamento das requisições
    - Envia os resultados para os respectivos clientes

Cliente-Servidor

- Identificação de Processos
  - Processos são identificados por portas
  - Cliente
    - Deve conhecer previamente a porta do servidor para tornar possível o envio de uma requisição
  - Servidor
    - Não precisa conhecer previamente a porta do cliente
    - Descobre a porta do cliente somente após receber uma requisição



Cliente-Servidor

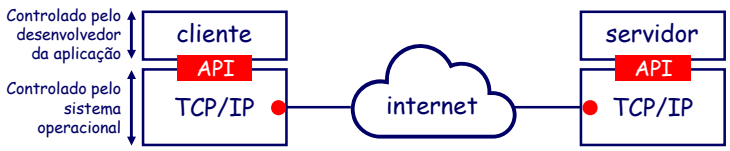
- Identificação de Processos
  - Portas são permanentemente reservadas para serviços padronizados e bem conhecidos
    - Portas UDP
      - Porta 53 (DNS)
      - Porta 161 (SNMP)
    - Portas TCP
      - Porta 25 (SMTP)
      - Porta 80 (HTTP)
  - Portas reservadas são utilizadas pelos servidores que implementam os respectivos serviços
  - Demais portas são disponíveis para os clientes

Cliente-Servidor

- Requisição de Portas
  - Cliente
    - Requisita uma porta qualquer não reservada
    - Sistema operacional escolhe uma porta qualquer que está disponível para clientes
  - Servidor
    - Requisita uma porta reservada e bem conhecida
      - Previamente reservada para o serviço
    - Informa ao sistema operacional a porta reservada que deseja utilizar

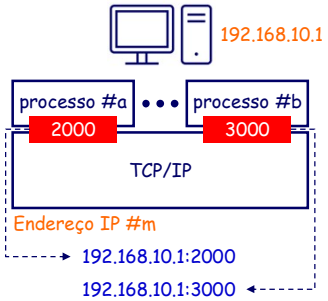
Sockets

- O que é um socket?
  - Representação interna do sistema operacional para um ponto de comunicação (*endpoint*) entre processos
  - Processos cliente e servidor realizam operações de leitura e escrita via API provida pelo sistema operacional



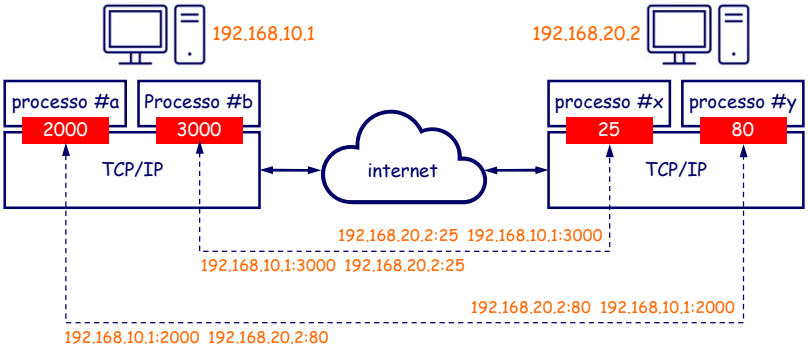
Sockets

- Endpoint
  - Representado pelo par endereço IP / porta
    - Endereço IP
      - Número de 32/128 bits que identifica globalmente cada dispositivo na inter-rede
    - Porta
      - Número inteiro (0 – 65535) que identifica localmente um ponto de comunicação no dispositivo
      - Identifica indiretamente o(s) processo(s) associado(s) ao ponto de comunicação



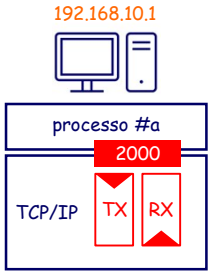
Sockets

- Socket Pair
  - Comunicação entre dois processos requer um par de endpoints
    - Endpoint local & Endpoint remoto



Sockets

- Buffers
  - Buffer de transmissão (TX)
    - Operação de escrita no socket armazena mensagem no buffer TX
    - Mensagem no buffer TX é enviada ao buffer RX do endpoint remoto
  - Buffer de recepção (RX)
    - Mensagem no buffer RX é recebida do buffer TX do endpoint remoto
    - Operação de leitura do socket recupera mensagem do buffer RX



Sockets

1

- Tipos de Sockets
  - Datagram Socket
    - Utiliza o serviço de datagrama não confiável e sem conexão provido pelo protocolo UDP
      - Trata a comunicação como um conjunto de mensagens individuais e independentes
      - Não garante a entrega e a sequência de datagramas
  - Stream Socket
    - Utiliza o serviço de circuito virtual confiável e orientado a conexão provido pelo protocolo TCP
      - Trata a comunicação como um fluxo contínuo de bytes entre cliente e servidor
      - Garante a entrega do fluxo de dados na sequência correta e sem erros

Sockets

- Socket UDP
  - Identificado apenas pelo endpoint local
    - Criado por default com valor 0.0.0.0:p
      - Aceita mensagens associadas a qualquer endereço IP do próprio dispositivo
    - Pode configurar com endereço IP específico
      - Aceita apenas mensagens associadas ao endereço IP específico configurado
    - Cliente adota porta alocada pelo sistema operacional
    - Servidor adota porta específica e bem conhecida

```
> netstat -uan
Proto  Recv-Q  Send-Q  Local Address      Foreign Address  State
udp         0       0  192.168.10.33:161  0.0.0.0:*
udp         0       0  192.168.10.65:161  0.0.0.0:*
udp         0       0  0.0.0.0:161        0.0.0.0:*
```

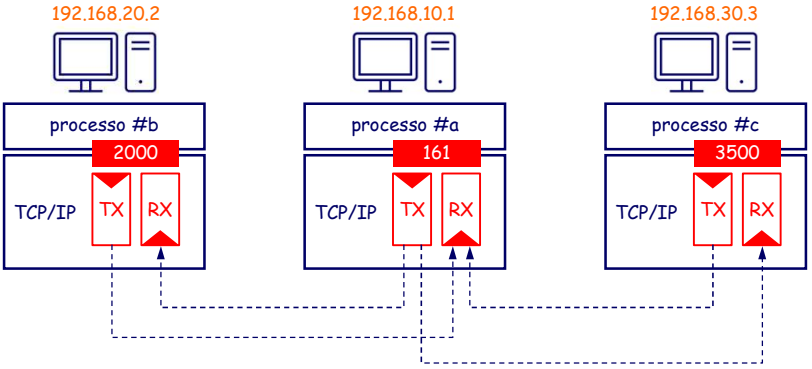

Sockets

- Socket UDP
  - Endpoint remoto não é requerido
    - Criado por default com valor 0.0.0.0:*
      - Envia mensagens a qualquer endereço IP e porta
    - Pode configurar com endereço IP/porta específica
      - Envia mensagens apenas ao endereço IP e porta específica configurada
    - Deve ser informado pelo cliente e servidor em cada mensagem a ser enviada
      - Exceto quando configurado com endereço IP/porta

```
> netstat -uan
Proto  Recv-Q  Send-Q  Local Address      Foreign Address  State
udp         0       0  192.168.10.33:161  0.0.0.0:*
udp         0       0  192.168.10.65:161  0.0.0.0:*
udp         0       0  0.0.0.0:161        0.0.0.0:*
```

Sockets

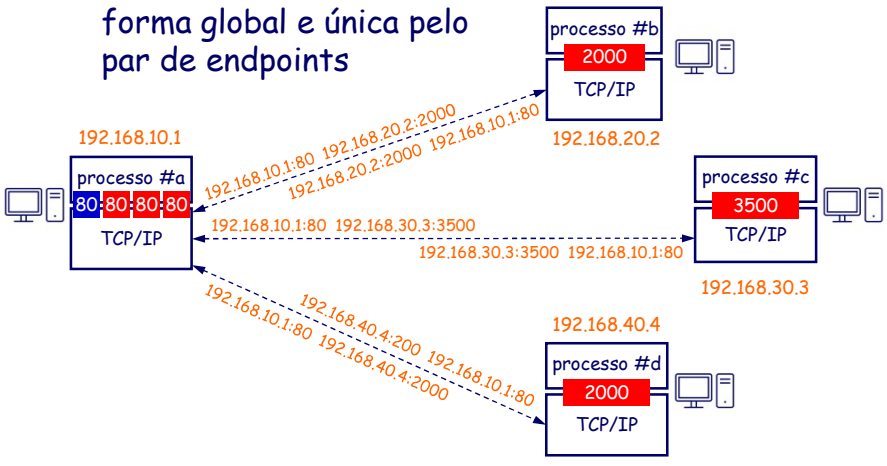
- Socket UDP
  - Buffers
    - Tratam mensagens associadas ao endpoint local e diversos endpoints remotos



23

Sockets

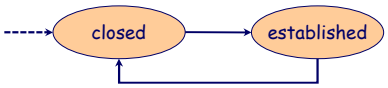
- Socket TCP
  - Conexão é identificada de forma global e única pelo par de endpoints



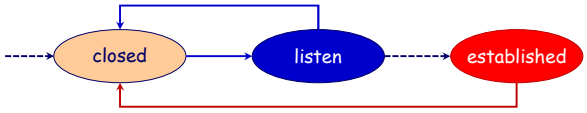
24

Sockets

- Socket TCP
  - Socket Ativo
    - Usado pelos clientes
    - Inicia requisição de conexão



  - Socket Passivo
    - Usado pelos servidores
    - Aguarda por requisição de conexão e permite a transferência de dados



25

Sockets

- Socket TCP
  - Socket Passivo
    - Socket de Acesso (welcoming socket)
      - Aguarda por requisição de conexão dos clientes
      - Sistema operacional cria o socket de conexão após aceitar a requisição de um dado cliente
    - Socket de Conexão (connection socket)
      - Permite a transferência de dados dedicada com o respectivo cliente conectado



26

Sockets

- Socket TCP
  - Socket Ativo
    - Endpoint local é atribuído pelo sistema operacional da estação cliente
      - Escolhe endereço IP do próprio dispositivo
      - Aloca porta qualquer disponível no dispositivo
    - Endpoint remoto é informado pelo processo cliente
      - Usuário ou programador deve fornecer o endereço IP/porta do servidor
      - Requisição de conexão é enviada ao socket de acesso no lado do servidor

```
> netstat -tan
Proto  Recv-Q  Send-Q  Local Address       Foreign Address  State
tcp         0       0  192.168.20.2:2000   192.168.10.1:80  ESTABLISHED
```


Sockets

- Socket TCP
  - Socket de Acesso
    - Endpoint local criado por default com valor 0.0.0.0:p
      - Aceita requisições de conexão associadas a qualquer endereço IP do próprio dispositivo
    - Pode configurar com endereço IP específico
      - Aceita apenas requisições de conexão associadas ao endereço IP específico configurado
    - Servidor adota porta específica e bem conhecida

```
> netstat -tan
Proto  Recv-Q  Send-Q  Local Address       Foreign Address  State
tcp         0       0  192.168.10.33:80    0.0.0.0:*        LISTEN
tcp         0       0  192.168.10.65:80    0.0.0.0:*        LISTEN
tcp         0       0  0.0.0.0:80          0.0.0.0:*        LISTEN
```

Sockets

- Socket TCP
  - Socket de Acesso
    - Endpoint remoto não é requerido
    - Criado por default com valor 0.0.0.0:*
      - Aceita requisições de conexão de clientes com qualquer endereço IP e porta
    - Não deve ser configurado com endereço IP/porta específica

```
> netstat -tan
Proto  Recv-Q  Send-Q  Local Address      Foreign Address  State
tcp         0       0  192.168.10.33:80   0.0.0.0:*        LISTEN
tcp         0       0  192.168.10.65:80   0.0.0.0:*        LISTEN
tcp         0       0  0.0.0.0:80         0.0.0.0:*        LISTEN
```

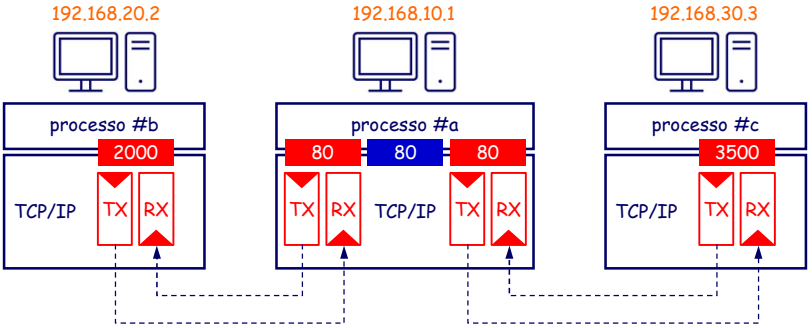

Sockets

- Socket TCP
  - Socket de Conexão
    - Endpoint local é atribuído pelo sistema operacional a partir da requisição de conexão do cliente
      - Adota endereço IP do servidor indicado pelo cliente
      - Herda a mesma porta local do socket de acesso
    - Endpoint remoto é obtido da requisição de conexão enviada pelo cliente

```
> netstat -tan
Proto  Recv-Q  Send-Q  Local Address     Foreign Address     State
tcp         0       0  0.0.0.0:80        0.0.0.0:*           LISTEN
tcp         0       0  192.168.10.1:80   192.168.20.2:2000   ESTABLISHED
```

Sockets

- Socket TCP
  - Buffers
    - Tratam apenas mensagens associadas aos endpoints local e remoto da respectiva conexão TCP



31

API Socket

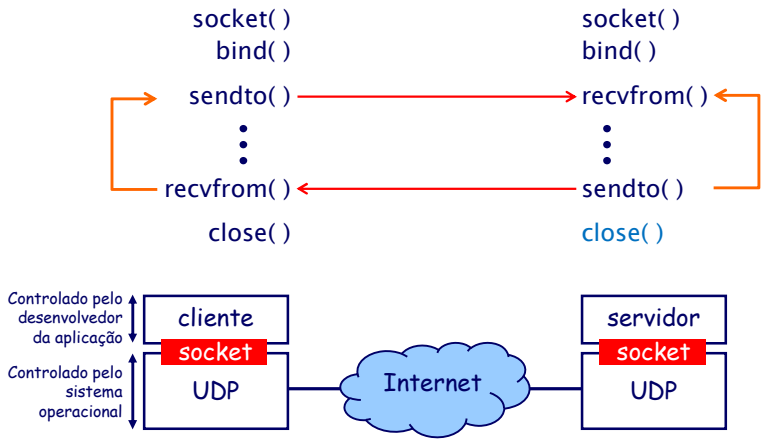
- Funções em C

Função	Descrição
socket	Cria um novo ponto de comunicação
bind	Associa endereço IP e porta ao endpoint local
listen	Habilita socket para aceitar requisições de conexões TCP
accept	Aguarda requisição de conexão TCP Bloqueia o processo chamador
connect	Estabelece conexão TCP com endpoint remoto
read	Ler dados de socket TCP conectado ao endpoint remoto
recvfrom	Ler dados de socket UDP
write	Envia dados no socket TCP conectado ao endpoint remoto
sendto	Envia dados no socket UDP ao endpoint remoto informado
close	Fecha socket

32

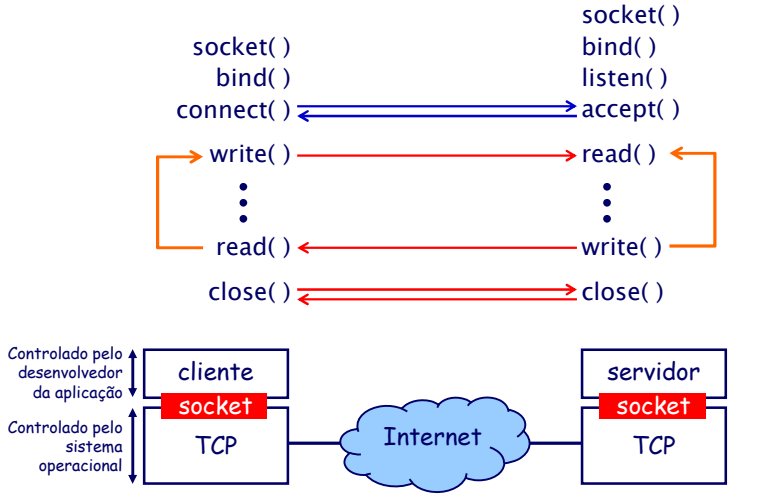
API Socket

- Socket UDP



API Socket

- Socket TCP



API Socket

- Classes em Java

Classe	Descrição
Socket(InetAddress, int)	Cria socket no cliente TCP Realiza as operações bind() e connect()
Socket(String, int)	Cria socket no cliente TCP Realiza as operações bind() e connect()
ServerSocket(int)	Cria socket no servidor TCP Realiza as operações bind() e listen() Provê o método accept()
DatagramSocket(int)	Cria socket no cliente/servidor UDP Realiza a operação bind()

Cliente/Servidor TCP

- Mensagem Simples

```
public class HelloClient {
  public static void main(String[] args) throws IOException {
    Socket s = new Socket("localhost", 4444);
    DataInputStream in = new DataInputStream(s.getInputStream());
    DataOutputStream out = new DataOutputStream(s.getOutputStream());
    out.writeUTF("Hello Server");
    System.out.println(in.readUTF());
    in.close();
    out.close();
    s.close();
  }
}
```

Cliente/Servidor TCP

- Mensagem Simples

```
public class HelloServer {
  public static void main(String[] args) throws IOException {
    ServerSocket s = new ServerSocket(4444);
    Socket ns = s.accept();
    DataInputStream in = new DataInputStream(ns.getInputStream());
    DataOutputStream out = new DataOutputStream(ns.getOutputStream());
    System.out.println(in.readUTF());
    out.writeUTF("Hello Client");
    in.close();
    out.close();
    ns.close();
    s.close();
  }
}
```


Cliente/Servidor TCP

- Protocolo Simples

```
public class ProtocolClient {
  public static void main(String[] args) throws IOException {
     socket s = new Socket("localhost", 4444);
    DataInputStream in = new DataInputStream(s.getInputStream());
    DataOutputStream out = new DataOutputStream(s.getOutputStream());
    Scanner stdIn = new Scanner(System.in);
    while (true) {
      String fromUser = stdIn.nextLine();
      out.writeUTF(fromUser);
      System.out.println(in.readUTF());
      if (fromUser.equals("Bye")) break;
    }
    in.close();
    out.close();
    s.close()
  }
}
```

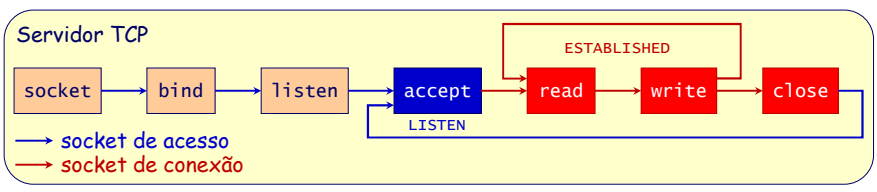
Cliente/Servidor TCP

- Protocolo Simples

```
public class ProtocolServer {
  public static void main(String[] args) throws IOException {
    ServerSocket s = new ServerSocket(4444);
    Socket ns = s.accept();
    DataInputStream in = new DataInputStream(ns.getInputStream());
    DataOutputStream out = new DataOutputStream(ns.getOutputStream());
    while (true) {
      String fromClient = in.readUTF();
      out.writeUTF(Protocol.processLine(fromClient));
      if (fromClient.equals("Bye")) break;
    }
    in.close();
    out.close();
    ns.close()
    s.close();
  }
}
```


Cliente/Servidor TCP

- Servidor Iterativo
  - Trata requisições de um único cliente a cada instante
    - Adequado para serviços com baixa taxa de requisições e/ou baixa carga de processamento
  - Implementado como um único processo
  - Também denominado single-threaded



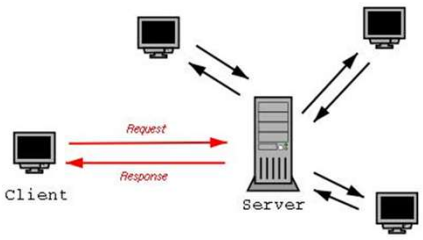
Cliente/Servidor TCP

- Servidor Iterativo

```
public class IterativeProtocolServer {
  public static void main(String[] args) throws IOException {
    ServerSocket s = new ServerSocket(4444);
    while (true) {
      Socket ns = s.accept();
      DataInputStream in = new DataInputStream(ns.getInputStream());
      DataOutputStream out = new DataOutputStream(ns.getOutputStream());
      while (true) {
        String fromClient = in.readUTF();
        out.writeUTF(Protocol.processLine(fromClient));
        if (fromClient.equals("Bye")) break;
      }
      in.close();
      out.close();
      ns.close()
    }
    s.close();
  }
}
```

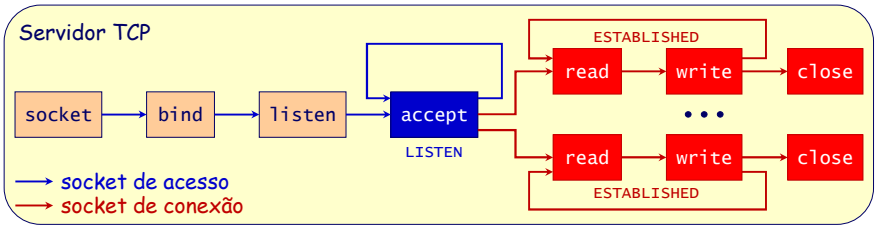

Cliente/Servidor TCP

- Servidor Concorrente
  - Servidor iterativo é capaz de responder apenas uma única requisição por vez
    - Função accept bloqueia o processo até receber uma nova requisição de conexão
  - Ideal é atender vários clientes concorrentemente



Cliente/Servidor TCP

- Servidor Concorrente
  - Trata requisições de vários clientes de forma simultânea
    - Adequado para serviços com alta taxa de requisições e/ou alta carga de processamento
  - Implementado como vários processos ou threads
  - Também denominado multi-threaded



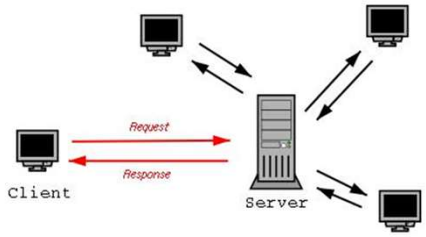
Cliente/Servidor TCP

- Servidor Concorrente
  - Existe um único socket de acesso no estado listen
    - Somente processa requisições de conexão
  - Podem existir diversos sockets de conexão no estado established
    - Somente processam segmentos de dados
  - Endpoints local e remoto do socket de acesso e sockets de conexão devem ser diferentes

```
> netstat -p tcp -a -n
Proto  Recv-Q  Send-Q  Local Address     Foreign Address     State
tcp         0       0  0.0.0.0:80        0.0.0.0:*           LISTEN
tcp         0       0  192.168.10.1:80   192.168.20.2:2000   ESTABLISHED
tcp         0       0  192.168.10.1:80   192.168.30.3:3500   ESTABLISHED
tcp         0       0  192.168.10.1:80   192.168.40.4:2000   ESTABLISHED
```

Cliente/Servidor TCP

- Servidor Concorrente
  - Instanciar um thread dedicado para cada cliente requisitante
    - Responsável pela interação com o cliente
  - Criar uma classe para tratar cada requisição implementando a interface Runnable



Cliente/Servidor TCP

- Servidor Concorrente

```
public class ProtocolServer implements Runnable {
  Socket s;

  public ProtocolServer(Socket ns) {
    s = ns;
  }

  public void run() {
    DataInputStream in = new DataInputStream(s.getInputStream());
    DataOutputStream out = new DataOutputStream(s.getOutputStream());
    while (true) {
      String fromClient = in.readUTF();
      out.writeUTF(protocol.processLine(fromClient));
      if (fromClient.equals("Bye")) break;
    }
    in.close();
    out.close();
    s.close();
  }
}
```

Cliente/Servidor TCP

- Servidor Concorrente

```
public class ThreadedServer {
  public static void main(String[] args) throws IOException {
    ServerSocket s = new ServerSocket(4444);
    while (true) {
      Socket ns = s.accept();
      Thread.startVirtualThread(new ProtocolServer(ns));
    }
    s.close();
  }
}
```


Cliente/Servidor TCP

- Servidor com Pool de Threads

```
public class CachedServer {
  public static void main(String[] args) throws IOException {
    Executor exec = Executors.newVirtualThreadPerTaskExecutor();
    ServerSocket s = new ServerSocket(4444);
    while (true) {
      Socket ns = s.accept();
      exec.execute(new ProtocolServer(ns));
    }
    s.close();
  }
}
```

Cliente/Servidor UDP

- Mensagem Simples

```
public class HelloClient {
   public static void main(String[] args) throws IOException {
      DatagramPacket in, out;
      DatagramSocket s = new DatagramSocket();
      byte[] outbuf = new String("Hello Server").getBytes();
      InetAddress server = InetAddress.getByName("localhost");
      out = new DatagramPacket(outbuf, outbuf.length, server, 4444);
      s.send(out);
      byte[] inbuf = new byte[256];
      in = new DatagramPacket(inbuf, inbuf.length);
      s.receive(in);
      System.out.println(new String(in.getData(), 0, in.getLength()));
      s.close();
   }
}
```


Cliente/Servidor UDP

- Mensagem Simples

```
public class HelloServer {
   public static void main(String[] args) throws IOException {
      DatagramPacket in, out;
      DatagramSocket s = new DatagramSocket(4444);
      byte[] inbuf = new byte[256];
      in = new DatagramPacket(inbuf, inbuf.length);
      s.receive(in);
      System.out.println(new String(in.getData(), 0, in.getLength()));
      InetAddress client = in.getAddress();
      int port = in.getPort();
      byte[] outbuf = new String("Hello Client").getBytes();
      out = new DatagramPacket(outbuf, outbuf.length, client, port);
      s.send(out);
      s.close();
   }
}
```

Cliente/Servidor UDP

- Protocolo Simples

```
public class ProtocolClient {
   public static void main(String[] args) throws IOException {
      Scanner stdin = new Scanner(System.in);
      DatagramPacket in, out;
      DatagramSocket s = new DatagramSocket();
      InetAddress server = InetAddress.getByName("localhost");
      while (true) {
         String fromUser = stdIn.nextLine();
         byte[] outbuf = fromUser.getBytes();
         out = new DatagramPacket(outbuf, outbuf.length, server, 4444);
         s.send(out);
         byte[] inbuf = new byte[256];
         in = new DatagramPacket(inbuf, inbuf.length);
         s.receive(in);
         System.out.println(new String(in.getData(), 0, in.getLength()));
         if (fromUser.equals("Bye")) break;
      }
      s.close();
   }
}
```


Cliente/Servidor UDP

- Protocolo Simples

```
public class ProtocolServer {
   public static void main(String[] args) throws IOException {
      DatagramPacket in, out;
      DatagramSocket s = new DatagramSocket(4444);
      while (true) {
         byte[] inbuf = new byte[256];
         in = new DatagramPacket(inbuf, inbuf.length);
         s.receive(in);
         String fromClient = new String(in.getData(), 0, in.getLength());
         InetAddress client = in.getAddress();
         int port = in.getPort();
         byte[] outbuf = Protocol.processLine(fromClient).getBytes();
         out = new DatagramPacket(outbuf, outbuf.length, client, port);
         s.send(out);
      }
      s.close();
   }
}
```

Cliente/Servidor UDP

- Servidor Concorrente

```
public class ProtocolServer implements Runnable {
   DatagramSocket s;
   DatagramPacket p;
   public ProtocolServer(DatagramSocket socket, DatagramPacket packet) {
      s = socket;
      p = packet;
   }
   public void run() {
      DatagramPacket out;
      String fromClient = new String(p.getData(), 0, p.getLength());
      InetAddress client = p.getAddress();
      int port = p.getPort();
      byte[] outbuf = protocol.processLine(fromClient).getBytes();
      out = new DatagramPacket(outbuf, outbuf.length, client, port);
      s.send(out);
   }
}
```


Cliente/Servidor UDP

- Servidor Concorrente

```
public class ThreadedServer {
   public static void main(String[] args) throws IOException {
      DatagramPacket in;
      DatagramSocket s = new DatagramSocket(4444);
      while (true) {
         byte[] inbuf = new byte[256];
         in = new DatagramPacket(inbuf, inbuf.length);
         s.receive(in);
         Thread.startVirtualThread(new ProtocolServer(s,in));
      }
      s.close();
   }
}
```

Cliente/Servidor UDP

- Servidor com Pool de Threads

```
public class CachedServer {
   public static void main(String[] args) throws IOException {
      DatagramPacket in;
      Executor exec = Executors.newVirtualThreadPerTaskExecutor();
      DatagramSocket s = new DatagramSocket(4444);
      while (true) {
         byte[] inbuf = new byte[256];
         in = new DatagramPacket(inbuf, inbuf.length);
         s.receive(in);
         exec.execute(new ProtocolServer(s,in));
      }
      s.close();
   }
}
```


Considerações Finais

- Pontos importantes
  - Threads também podem ser usados no lado do processo cliente
  - Muitas erros podem acontecer na comunicação em rede entre clientes e servidores
    - Não é possível resolver o nome do servidor
    - Host não pode ser encontrado - offline
    - Conexão pode ser encerrada abruptamente
    - Servidor pode demorar demais para responder
    - Inundação de requisições pode consumir recursos com inúmeros threads

Considerações Finais

- Pontos importantes
  - Trate exceções associadas a erros de comunicação
  - Defina o tempo máximo de bloqueio das operações de leitura com método `setSoTimeout()`
    - Gera exceção `SocketTimeOutException` após tempo expirar

```
try (Socket s = new Socket("www.google.com", 80)) {
   s.setSoTimeout(60000);
   DataInputStream in = ...;
   DataOutputStream out = ...;
   ...
   String linha = in.readUTF();
   ...
} catch (SocketTimeoutException e) {
   System.err.println("Erro: O tempo limite expirou!");
} catch (Exception e) {
   e.printStackTrace();
}
```


Considerações Finais

3

- Pontos importantes
  - Como fazer para o servidor interpretar e processar uma requisição do cliente?
  - Observe o comando Bye ...
  - Lembram do protocolo?
    - Define regras para interpretar os dados trocados em requisições e respostas